FATEC COTIA · ENGENHARIA DE SOFTWARE II

IFO — Organizador Financeiro Inteligente

Documentação do projeto · V1 revisada (correção e refatoração)
Escopo, requisitos, metodologia, modelo de dados, diagramas, roadmap e publicação

Equipe de desenvolvimento
João Marcos Vilaça Marques

Equipe de apresentação
João Marcos Vilaça Marques · Amanda Freitas de Lima
Lucas de Souza Saldanha dos Santos · Vinicius de Amorim Farias

Curso Desenvolvimento de Software Multiplataforma
Disciplina Engenharia de Software II
Período 2º Semestre
Aplicação no ar ifo-organizador-financeiro-frontend.vercel.app

Uso de IA desenvolvido com o apoio do Claude Code (Anthropic);
decisões, revisão e controle do Git são do desenvolvedor

Sumário

01 · Escopo do Projeto

1. Visão
2. Objetivos
3. Dentro do escopo (V1 revisada)
4. Fora do escopo (decisões já tomadas)
5. Decisões recentes de escopo
6. Premissas e restrições
7. Partes interessadas
8. Critérios de sucesso
9. Riscos

02 · Requisitos e Regras de Negócio

1. Requisitos funcionais
2. Requisitos não funcionais
3. Regras de negócio
4. Categorias fixas
5. Rastreabilidade resumida

03 · Metodologia de Desenvolvimento

1. Abordagem
2. Fluxo de trabalho com Git
3. Definition of Ready (pronto para começar)
4. Definition of Done (pronto para entregar)
5. Qualidade e testes
6. Arquitetura e padrões

04 · Modelo de Dados

1. Decisões do modelo
2. Diagrama entidade-relacionamento

05 · Diagramas

1. Diagrama de casos de uso
2. Diagrama de classes
3. Diagramas de sequência
4. Diagramas de processo (BPMN simplificado)
5. Arquitetura em camadas

06 · Roadmap, Backlog e Decisões

1. Roadmap
2. Backlog
3. Decisões de arquitetura
4. Publicação na web
5. Próxima versão (IFO V2)

01 · Escopo do Projeto

Projeto: IFO — Organizador Financeiro Inteligente (V1 revisada: correção e refatoração da V1)
Equipe de desenvolvimento: João Marcos Vilaça Marques
Equipe de apresentação: João Marcos Vilaça Marques, Amanda Freitas de Lima, Lucas de Souza Saldanha dos Santos, Vinicius de Amorim Farias
Uso de IA: desenvolvido por João Marcos com o apoio do Claude Code (Anthropic) para código, testes e documentação; decisões, revisão e controle do Git são do desenvolvedor.
Instituição: FATEC Cotia — Desenvolvimento de Software Multiplataforma (Engenharia de Software II)

---

1. Visão

O IFO é uma aplicação web que ajuda pessoas a controlar as finanças pessoais de forma simples: registrar receitas e despesas, seguir a regra 50/30/20, acompanhar metas e simular investimentos.

Problema: grande parte da população tem dificuldade para controlar gastos, o que leva a endividamento e falta de planejamento. Ferramentas existentes costumam ser complexas demais para quem está começando.

Proposta: um sistema leve, em português, com poucas telas e regras claras, que orienta o usuário pela divisão da renda em **Necessidades (50%)**, **Lazer (30%)** e **Investimentos (20%)** e avisa quando ele estoura um limite.

2. Objetivos

Geral: entregar um sistema web de gestão financeira pessoal com qualidade de engenharia (arquitetura em camadas, segurança, testes e documentação).

Específicos

1. Corrigir os problemas estruturais da V1 (segurança, inconsistência entre banco e código, regras faltantes).
2. Reduzir o escopo ao que realmente é usado (remover Conta/Carteira e persistência de Investimento).
3. Implementar o modelo **Grupo → Categoria (fixa) → Descrição (livre)** nas transações.
4. Permitir que o usuário **defina o próprio orçamento** (padrão 50/30/20, editável) e receba **alerta simples de estouro**.
5. Entregar interface **responsiva (mobile-first)** com design renovado.
6. Manter a documentação (UML, BPMN, requisitos) sincronizada com o código.

3. Dentro do escopo (V1 revisada)

#	Funcionalidade	Resumo
1	Cadastro e login	Autenticação segura (BCrypt + JWT)
2	Perfil	Editar nome e renda mensal
3	Transações	CRUD de receitas e despesas com categoria fixa + descrição livre
4	Filtro por mês	Listagem e totais por mês/ano
5	Resumo financeiro	Saldo, total de receitas e total de despesas do mês
6	Orçamento 50/30/20	Percentuais por grupo, editáveis pelo usuário (soma = 100%)

#	Funcionalidade	Resumo
7	Alerta de estouro	Aviso simples em 80% (atenção) e acima de 100% (estouro)
8	Metas financeiras	CRUD, aporte manual e barra de progresso
9	Simulador de investimentos	Juros simples e compostos, **sem salvar** no banco
10	Gráfico de pizza simples	Distribuição das despesas do mês por grupo
11	Despesas recorrentes	Modelo mensal (ex.: internet, luz) que gera a transação automaticamente
12	Exportar CSV	Download das transações do mês filtrado
13	Dark mode	Tema claro/escuro alternável, com padrão do sistema

4. Fora do escopo (decisões já tomadas)

- **Conta/Carteira** (tabela, API e diagramas) — removida; não será usada.
- **Persistência de simulações de investimento** — o simulador é apenas cálculo.
- **Categorias criadas pelo usuário** — a lista de categorias é fixa; o usuário altera apenas a **descrição** da transação.
- **Dashboards interativos** (gráficos dinâmicos, cartões avançados, drill-down) — ficam para o **IFO V2 (futuro)**.
- **Autenticação em dois fatores (2FA)** — também fica para o **IFO V2 (futuro)**.
- Integração bancária, app mobile e sugestões com IA — **descartados**: nem a V2 terá.

5. Decisões recentes de escopo

Em **18/10/2026** a equipe aprovou a inclusão de **despesas recorrentes**, **exportar CSV** e **dark mode** (prioridade *Should*: entram depois dos itens *Must*). Os detalhes estão em `02-requisitos.md` (RF20–RF22, RN14–RN19) e no roadmap (`07-roadmap.md`).

Em **31/10/2026**, concluídas as sprints R0–R5, entrou a **R6: publicar o IFO V1 na web** (link próprio e banco na nuvem, a custo zero), sem nenhuma funcionalidade nova. A escolha de hospedagem está no ADR-018 do `07-roadmap.md` .

Também em **31/10/2026** ficaram definidos dois pontos: o **desenvolvimento é individual** (João Marcos), enquanto a **apresentação é feita pelos 4 integrantes**; e o **IFO V2** não terá integração bancária, app mobile nem sugestões com IA. As diferenças principais da V2 são o **2FA por e-mail**, o **dashboard interativo** (Chart.js) e a troca de stack (ver `07-roadmap.md` , seção 4).

> Regra: qualquer item novo só entra na sprint com decisão registrada em `docs/` (ver ADRs no fim do `07-roadmap.md`).

6. Premissas e restrições

- Usuário único por conta (sem compartilhamento entre pessoas).
- Moeda: Real (BRL). Datas no formato brasileiro na interface.
- Stack mantida: Java 17 + Spring Boot 3.2, PostgreSQL 15, HTML/CSS/JavaScript puro, Docker para o banco.
- Desenvolvimento individual (João Marcos), sprints semanais, prazo acadêmico; a apresentação é feita pelos 4 integrantes da equipe.
- O frontend **não** usa frameworks pesados (decisão da V1 mantida para leveza e controle).

7. Partes interessadas

Papel	Quem	Interesse
Usuário final	Pessoas que querem organizar o dinheiro	Simplicidade e clareza
Equipe de desenvolvimento	1 aluno (João Marcos)	Aprendizado e entrega
Equipe de apresentação	4 alunos	Apresentar e defender o projeto no seminário
Professor(a) de Engenharia de Software II	FATEC Cotia	Aplicação de métodos e documentação

8. Critérios de sucesso

- Todos os requisitos "Must" de `02-requisitos.md` implementados e testados.
- Nenhuma senha em texto puro; nenhum endpoint expõe dados de outro usuário.
- Banco criado do zero apenas por migrations (sem scripts manuais) e aplicação sobe com um comando.
- Telas utilizáveis de 360 px (celular) a desktop.
- Documentação e diagramas coerentes com o código entregue.

9. Riscos

Risco	Impacto	Mitigação
Divergência código × documentação (como na V1)	Alto	Diagramas em Mermaid dentro do repo, revisados no Definition of Done
Escopo crescer (itens do IFO V2 invadirem a V1 revisada)	Médio	Lista "fora do escopo" + ADRs
Tempo curto	Médio	Priorização MoSCoW e sprints curtas
Problemas de ambiente (Docker/CORS)	Baixo	`docker-compose` + variáveis de ambiente documentadas

02 · Requisitos e Regras de Negócio

Prioridade segue MoSCoW: **M** (Must), **S** (Should), **C** (Could).

1. Requisitos funcionais

Conta do usuário

ID	Requisito	Prior.
RF01	O usuário se cadastra informando nome, e-mail, senha e renda mensal.	M
RF02	O usuário faz login com e-mail e senha e recebe um token JWT.	M
RF03	O usuário encerra a sessão (logout) no cliente.	M
RF04	O usuário edita nome e renda mensal.	M

Transações

ID	Requisito	Prior.
RF05	O usuário cadastra transação informando valor, data, **categoria** (lista fixa) e **descrição** livre. O tipo (receita/despesa) e o grupo são derivados da categoria.	M
RF06	O usuário lista, edita e exclui suas transações. A edição é um `PUT` (não "apagar e recriar").	M
RF07	O usuário filtra transações e totais por **mês/ano**.	M
RF08	O sistema exibe o resumo do mês: total de receitas, total de despesas e saldo.	M
RF09	O sistema lista as categorias disponíveis (somente leitura), agrupadas.	M
RF10	O sistema exibe **gráfico de pizza simples** das despesas do mês por grupo.	S

Orçamento

ID	Requisito	Prior.
RF11	Ao se cadastrar, o usuário recebe o orçamento padrão 50/30/20.	M
RF12	O usuário edita os percentuais de Necessidades, Lazer e Investimentos; a soma deve ser 100%.	M
RF13	O sistema calcula, por grupo, limite (R$), gasto no mês, % usado e nível (OK / ATENÇÃO / ESTOURO).	M
RF14	Ao lançar uma despesa, o sistema avisa se o grupo entrou em ATENÇÃO ou ESTOURO. O aviso também aparece no dashboard.	M

Metas

ID	Requisito	Prior.
RF15	O usuário cria, lista, edita e exclui metas (título, valor objetivo, prazo).	M
RF16	O usuário registra aportes na meta (atualiza o valor atual).	M
RF17	O sistema exibe barra de progresso (%) e marca a meta como concluída ao atingir o objetivo.	M

Simulador de investimentos

ID	Requisito	Prior.
RF18	O usuário informa aplicação inicial (opcional), aporte mensal, taxa mensal (%), prazo (meses) e tipo de juros (simples/composto) e recebe montante final, total investido e rendimento.	M
RF19	O resultado traz a evolução mês a mês em tabela. **Nada é salvo no banco.**	S

Recursos complementares (aprovados em 18/10/2026)

ID	Requisito	Prior.
RF20	**Despesas recorrentes:** o usuário cadastra, lista, edita, pausa e exclui um modelo mensal (categoria de despesa, valor, descrição, dia do mês, início e fim opcional). O sistema gera a transação automaticamente a cada mês.	S
RF21	**Exportar CSV:** o usuário baixa as transações do mês/ano filtrado em arquivo CSV.	S
RF22	**Dark mode:** o usuário alterna entre tema claro e escuro; o padrão segue o sistema e a escolha fica salva no navegador.	S

2. Requisitos não funcionais

ID	Categoria	Requisito
RNF01	Segurança	Senhas com **BCrypt**; autenticação **JWT** stateless; a API nunca devolve a senha/hash.
RNF02	Segurança	O identificador do usuário vem do **token**, nunca de parâmetro de URL (`?usuario=`). Cada usuário só acessa os próprios dados.
RNF03	Segurança	Entrada validada (Bean Validation) e saída escapada no frontend (sem `innerHTML` com dado do usuário).
RNF04	Segurança	Segredos (senha do banco, chave JWT) por **variáveis de ambiente**, nunca no repositório. CORS restrito à origem do frontend.
RNF05	Desempenho	Listagens respondem em menos de 2 s para 1.000 transações do usuário.
RNF06	Usabilidade	Fluxo "lançar despesa" em no máximo 4 interações; mensagens de erro claras em português.
RNF07	Responsividade	Layout **mobile-first**, funcional a partir de 360 px de largura até desktop.
RNF08	Acessibilidade	Contraste mínimo AA, campos com `label` , navegação por teclado, foco visível.
RNF09	Manutenibilidade	Camadas Controller → Service → Repository; **DTOs** na API; entidades não expostas.
RNF10	Manutenibilidade	Esquema do banco versionado por **Flyway**; `ddl-auto=validate` .
RNF11	Confiabilidade	Valores monetários em `BigDecimal` / `NUMERIC(12,2)` ; erros tratados por `@ControllerAdvice` com formato único.
RNF12	Testabilidade	Testes unitários de serviços e BDD (Cucumber) chamando o código real; banco de teste isolado (Testcontainers).
RNF13	Portabilidade	`docker-compose` sobe banco (e opcionalmente backend); README com passo a passo.
RNF14	Compatibilidade	Chrome, Edge, Firefox e Safari atuais.
RNF15	Segurança	A exportação CSV usa apenas dados do usuário do token e neutraliza injeção de fórmulas (RN18).

ID	Categoria	Requisito
RNF16	Acessibilidade	Os dois temas (claro e escuro) mantêm contraste AA; cores definidas só por variáveis CSS.
RNF17	Confiabilidade	A geração de recorrências é idempotente: rodar o job duas vezes não duplica transações (RN15).

3. Regras de negócio

ID	Regra
RN01	O valor de uma transação é sempre **> 0**; o sinal vem do tipo (receita soma, despesa subtrai).
RN02	O **tipo** (RECEITA/DESPESA) e o **grupo** (NECESSIDADES/LAZER/INVESTIMENTOS) da transação são **derivados da categoria**. O cliente envia apenas a categoria.
RN03	Receitas não pertencem a grupo de orçamento.
RN04	**Saldo = Σ receitas − Σ despesas.** Aportes em Investimentos contam como despesa (saem do saldo disponível).
RN05	Orçamento: os três percentuais são ≥ 0 e **somam exatamente 100%**. Padrão: 50 / 30 / 20.
RN06	**Limite do grupo = renda mensal × percentual.** Se a renda for 0, nenhum limite é calculado e o aviso não é exibido (o app pede para informar a renda).
RN07	Gasto do grupo = Σ despesas do mês/ano no grupo. **% usado = gasto ÷ limite.** Níveis: **OK** < 80%, **ATENÇÃO** de 80% a 100%, **ESTOURO** > 100%.
RN08	O alerta é informativo: **nunca bloqueia** o lançamento da despesa.
RN09	Meta: progresso = valor atual ÷ valor objetivo (exibido limitado a 100%). Concluída quando valor atual ≥ objetivo. Objetivo > 0; valor atual ≥ 0.
RN10	E-mail único; senha com no mínimo 8 caracteres; renda mensal ≥ 0.
RN11	Um usuário só lê, altera ou exclui dados que lhe pertencem (transações e metas).
RN12	Categorias são **fixas** (carregadas por migration) e iguais para todos os usuários.

Fórmulas do simulador (RN13)

Seja `P` a aplicação inicial, `PMT` o aporte mensal (feito no **fim** de cada mês), `i` a taxa mensal em decimal (`taxa% ÷ 100`) e `n` o prazo em meses.

- **Composto:** `M = P·(1+i)^n + PMT · ((1+i)^n − 1) / i` (se `i = 0`, `M = P + PMT·n`)
- **Simples:** `M = P·(1 + i·n) + PMT·n + PMT·i·n·(n−1)/2`
- **Total investido** = `P + PMT·n`; **rendimento** = `M − total investido`.

Conferência (exemplo da V1): `P=0`, `PMT=300`, `i=1%`, `n=12`, composto → **R$ 3.804,75**.

Arredondamento: `HALF_EVEN` / `HALF_UP` com 2 casas apenas no resultado final.

Regras de recorrência, exportação e tema

ID	Regra
RN14	Só categorias do tipo **DESPESA** podem ser recorrentes. O dia do mês vai de 1 a 31; em meses mais curtos vale o **último dia** do mês.

ID	Regra
RN15	Um job diário (`@Scheduled`) gera as ocorrências cuja data já chegou. Cada recorrência guarda a **última competência gerada** (mês); a geração avança mês a mês até o mês atual, inclusive meses perdidos, sem duplicar. Ao criar uma recorrência, as ocorrências já vencidas são geradas na hora.
RN16	Editar, pausar ou excluir uma recorrência afeta **somente meses futuros**. Transações já geradas permanecem, são transações comuns (editáveis e excluíveis) e continuam contando no orçamento. Excluir a recorrência mantém as transações geradas (vínculo vira nulo).
RN17	Recorrência só gera dentro do período (`data_inicio` ≤ ocorrência ≤ `data_fim` , se houver) e enquanto estiver **ativa**.
RN18	**CSV:** colunas `Data;Tipo;Grupo;Categoria;Descrição;Valor` ; separador `;` ; valor com vírgula decimal e sem símbolo de moeda; datas `dd/MM/yyyy` ; codificação UTF-8 **com BOM** (abre certo no Excel). Textos iniciados por `=` , `+` , `-` ou `@` recebem apóstrofo na frente. Exporta só as transações do usuário no mês/ano pedido.
RN19	**Tema:** sem escolha do usuário, vale o `prefers-color-scheme` do sistema. A escolha manual é salva em `localStorage` e prevalece.

4. Categorias fixas

Tipo	Grupo	Categorias
RECEITA	—	Salário · Freelance · Rendimentos · Mesada/Presente · Reembolso · Outras receitas
DESPESA	NECESSIDADES	Moradia · Contas fixas · Alimentação · Transporte · Saúde · Educação · Parcelas e dívidas
DESPESA	LAZER	Delivery · Passeio · Assinaturas · Compras e vestuário · Viagem · Outros (lazer)
DESPESA	INVESTIMENTOS	Renda fixa · Renda variável · Reserva de emergência · Previdência

Exemplos de uso (campo **descrição** livre):

- *Delivery* → "pizza"
- *Contas fixas* → "internet", "luz"
- *Assinaturas* → "streaming"

5. Rastreabilidade resumida

Funcionalidade	Requisitos	Regras
Cadastro/Login	RF01–RF04, RNF01–RNF04	RN10, RN11
Transações	RF05–RF10	RN01–RN04, RN12
Orçamento e alerta	RF11–RF14	RN05–RN08
Metas	RF15–RF17	RN09
Simulador	RF18–RF19	RN13
Recorrências	RF20	RN14–RN17
Exportar CSV	RF21	RN18
Dark mode	RF22	RN19

03 · Metodologia de Desenvolvimento

1. Abordagem

Continuamos com **desenvolvimento ágil em sprints semanais** (Scrum leve), como na V1. O trabalho é incremental: cada sprint termina com algo que roda e pode ser demonstrado.

Elemento	Como fazemos
Duração da sprint	1 semana
Planejamento	No início: escolher itens do backlog (GitHub Projects) e definir a meta da sprint
Acompanhamento	Quadro Kanban: `Backlog → A fazer → Em andamento → Em revisão → Pronto`
Revisão/retrospectiva	No fim: demonstrar o incremento, listar o que funcionou e o que ajustar
Priorização	MoSCoW (ver `02-requisitos.md`)
Papéis	Desenvolvimento individual (João Marcos): ele planeja, codifica, testa e revisa a própria sprint. A equipe de 4 participa da apresentação

2. Fluxo de trabalho com Git

- Branch principal: `master` (sempre funcional).
- Branches curtas por tarefa: `feature/<resumo>`, `fix/<resumo>`, `docs/<resumo>`.
- **Pull Request obrigatório** para entrar na `master`, com **autorrevisão** do diff pelo desenvolvedor e a suíte de testes verde (não há revisão de outro membro).
- **Commits convencionais** (em português, imperativo):
  - `feat: adiciona filtro de transações por mês`
  - `fix: corrige cálculo do saldo`
  - `docs: atualiza diagrama de classes`
  - `refactor:`, `test:`, `chore:`
- Cada tarefa do quadro vira uma *issue*; o PR referencia a issue (`Closes #12`).

3. Definition of Ready (pronto para começar)

Uma tarefa só entra na sprint se tiver: objetivo claro, requisitos/regras relacionados (IDs `RF` / `RN`), critério de aceite e estimativa (P/M/G).

4. Definition of Done (pronto para entregar)

- [ ] Código compila e a aplicação sobe localmente.
- [ ] Testes novos/ajustados passam (`./mvnw test`).
- [ ] Regras de negócio da tarefa cobertas por teste.
- [ ] Sem segredos, senhas ou dados reais no código.
- [ ] Autorrevisão do PR (diff conferido pelo desenvolvedor, `git status` sem arquivos inesperados).
- [ ] **Documentação atualizada** (requisitos, API e diagramas Mermaid, se afetados).
- [ ] Interface verificada em celular (≈360 px) e desktop, quando houver tela.

5. Qualidade e testes

Nível	Ferramenta	O que cobre
Unitário	JUnit 5 + Mockito	Regras de negócio nos *services* (saldo, alerta, juros, metas)
Integração	Spring Boot Test + **Testcontainers (PostgreSQL)**	Repositórios, migrations e endpoints protegidos
Aceitação (BDD)	Cucumber (português)	Fluxos: cadastro/login, lançar despesa com alerta, meta, simulador
Manual	Roteiro por tela	Responsividade, acessibilidade básica, bugs visuais

Regra: os passos do Cucumber **chamam o código real** (services/API). Nada de "login simulado" com senha fixa, como na V1.

6. Arquitetura e padrões

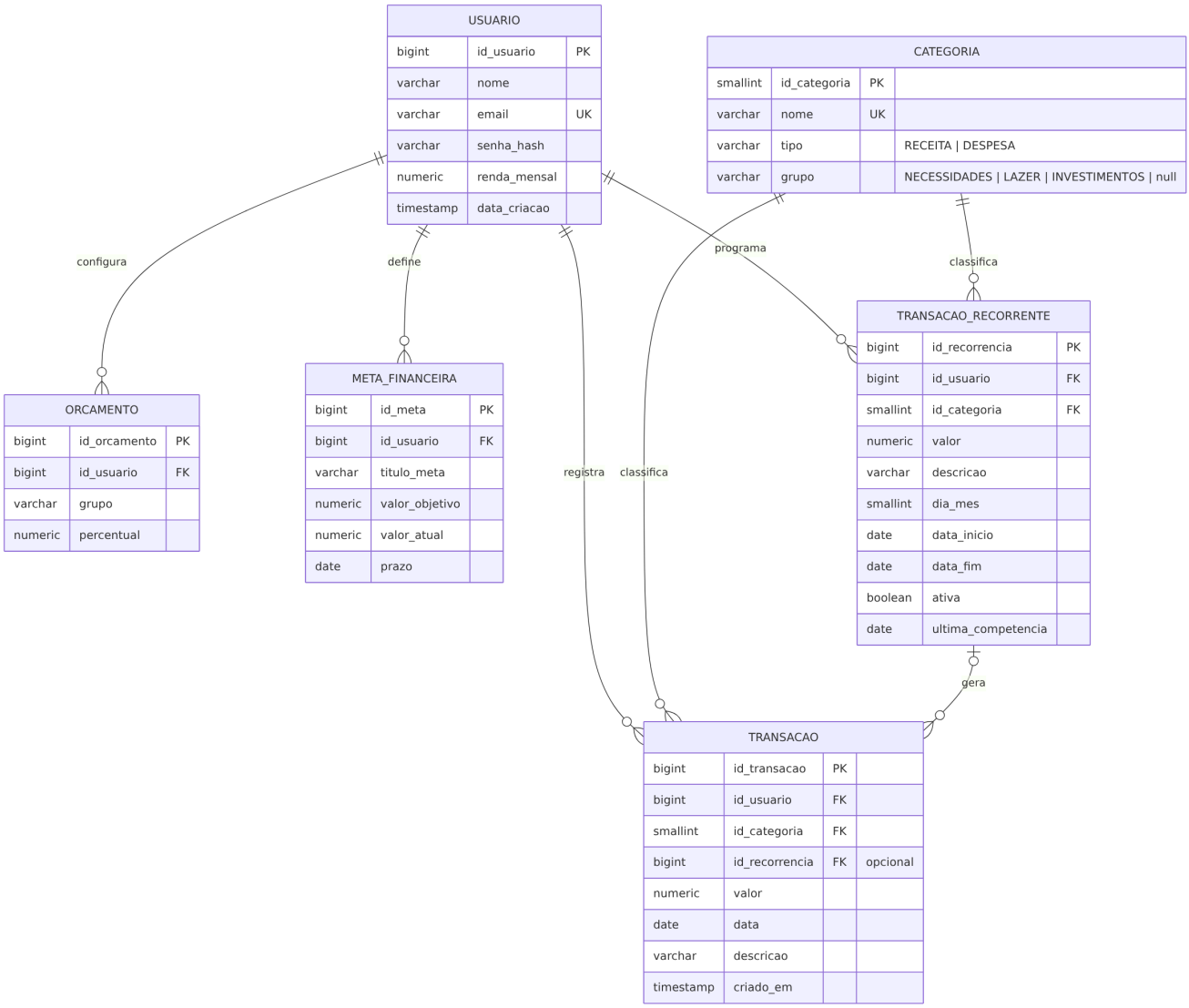
- **Backend:** MVC em camadas — `controller` (REST/DTO) → `service` (regras) → `repository` (Spring Data JPA) → `model` (entidades).
- **Segurança:** Spring Security, filtro JWT, `PasswordEncoder` BCrypt.
- **Erros:** `@RestControllerAdvice` com resposta padronizada (`status` , `mensagem` , `campos`).
- **Banco:** PostgreSQL com migrations **Flyway** (`db/migration/V1__...sql`).
- **Frontend:** HTML5 + CSS3 (variáveis CSS, mobile-first) + JavaScript modular (ES modules), sem framework.
- **Configuração:** `application.properties` lê variáveis de ambiente (`DB_URL` , `DB_USER` , `DB_PASSWORD` , `JWT_SECRET`).

04 · Modelo de Dados

1. Decisões do modelo

- **Sai:** `conta_carteira` e `investimento` (o simulador não persiste nada).
- **Categoria é fixa e global** (sem `id_usuario`), carregada por migration. Ela carrega o **tipo** e o **grupo**.
- **Transação não guarda o tipo nem o grupo**: ambos vêm da categoria (evita inconsistência, regra RN02).
- **Orçamento** é a configuração de 3 percentuais por usuário (um por grupo). O limite em R$ é calculado: `renda × percentual`.
- **Recorrência** é um modelo (`transacao_recorrente`) que gera transações comuns; a transação gerada guarda `id_recorrencia` (opcional) e depois vive sozinha.
- Valores monetários: `NUMERIC(12,2)`. Percentuais: `NUMERIC(5,2)`.
- Esquema criado **somente por migrations Flyway**; JPA com `ddl-auto=validate`.

2. Diagrama entidade-relacionamento

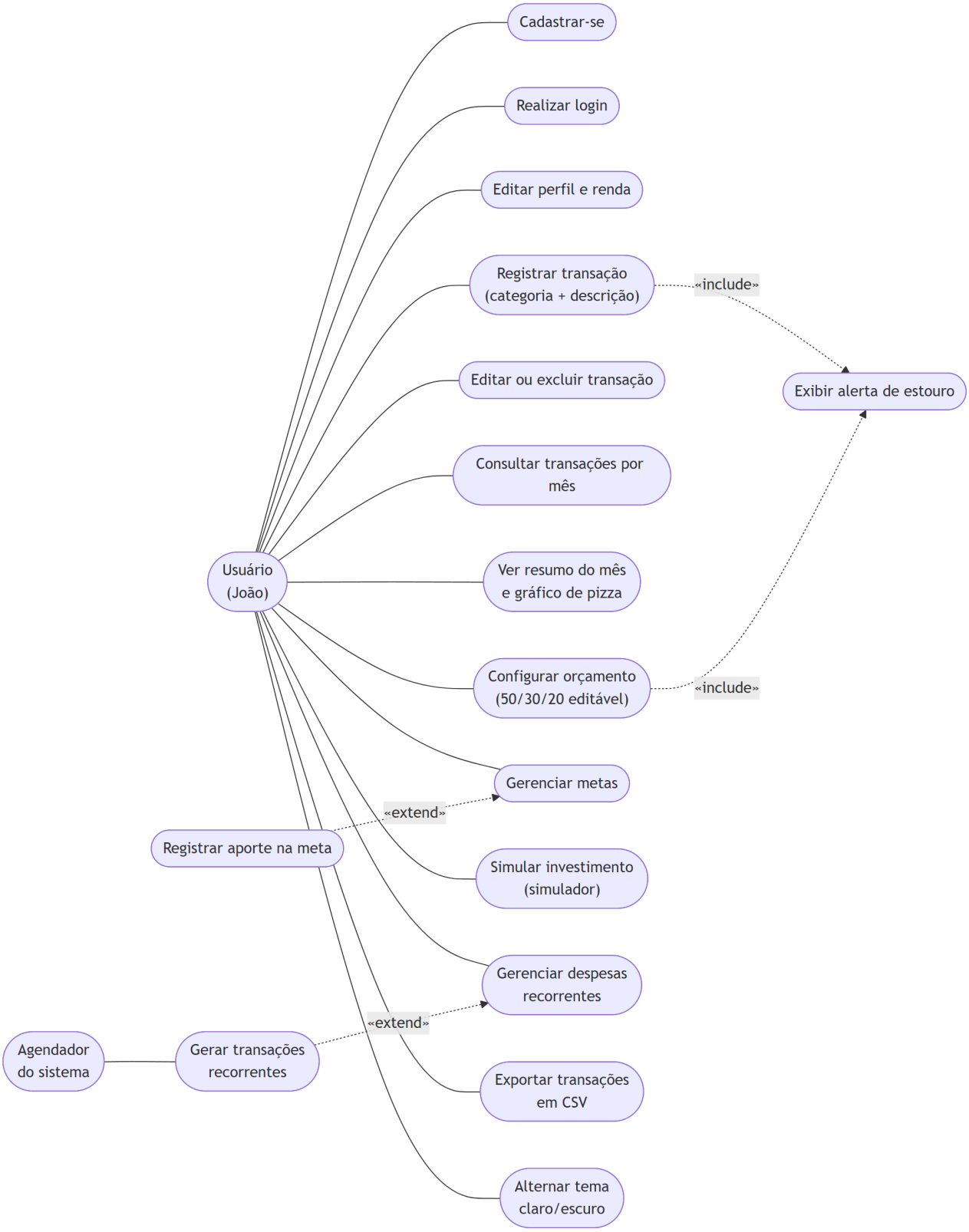


05 · Diagramas

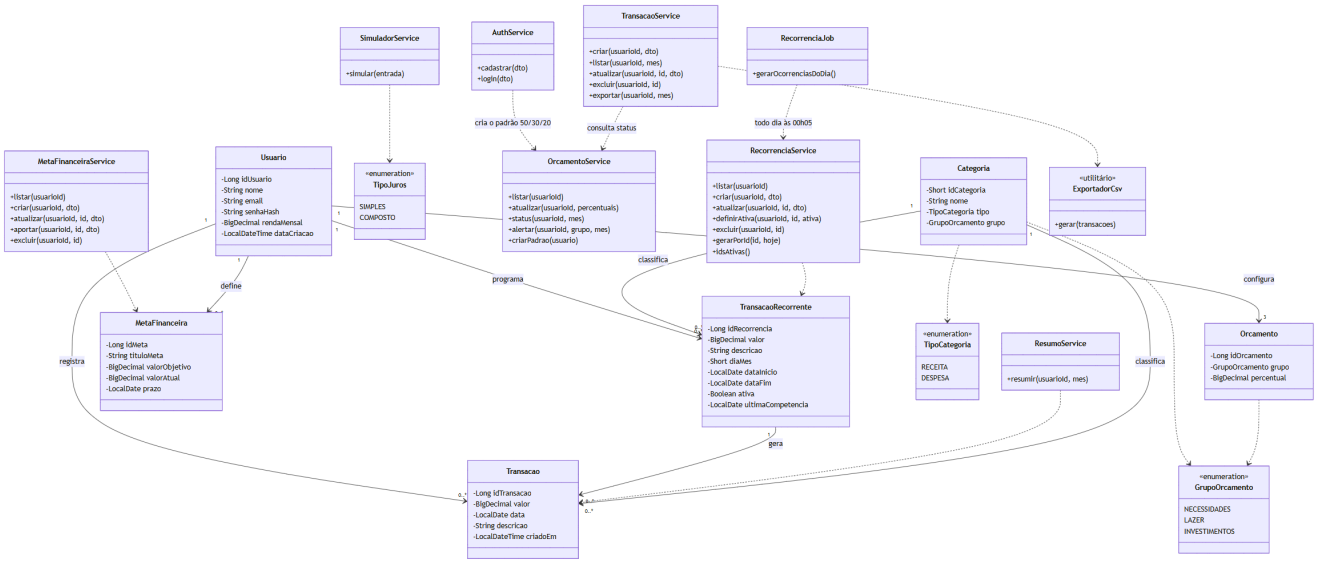
Todos os diagramas estão em **Mermaid** (renderizam direto no GitHub e no VS Code com extensão). O usuário de exemplo é **João**. Eles substituem os diagramas da V1 e refletem o escopo da V1 revisada (sem Conta/Carteira, sem persistência de investimento).

1. Diagrama de casos de uso

Todos os casos de uso abaixo fazem parte do sistema **IFO**; o único ator é o usuário (João).


Cadastrar-se
Realizar login
Editar perfil e renda
Registrar transação (categoria + descrição)
«include»
Editar ou excluir transação
Exibir alerta de estouro
Consultar transações por mês
Usuário (João)
Ver resumo do mês e gráfico de pizza
Configurar orçamento (50/30/20 editável)
«include»
Gerenciar metas
«extend»
Registrar aporte na meta
Simular investimento (simulador)
Gerenciar despesas recorrentes
«extend»
Agendador do sistema
Gerar transações recorrentes
Exportar transações em CSV
Alternar tema claro/escuro

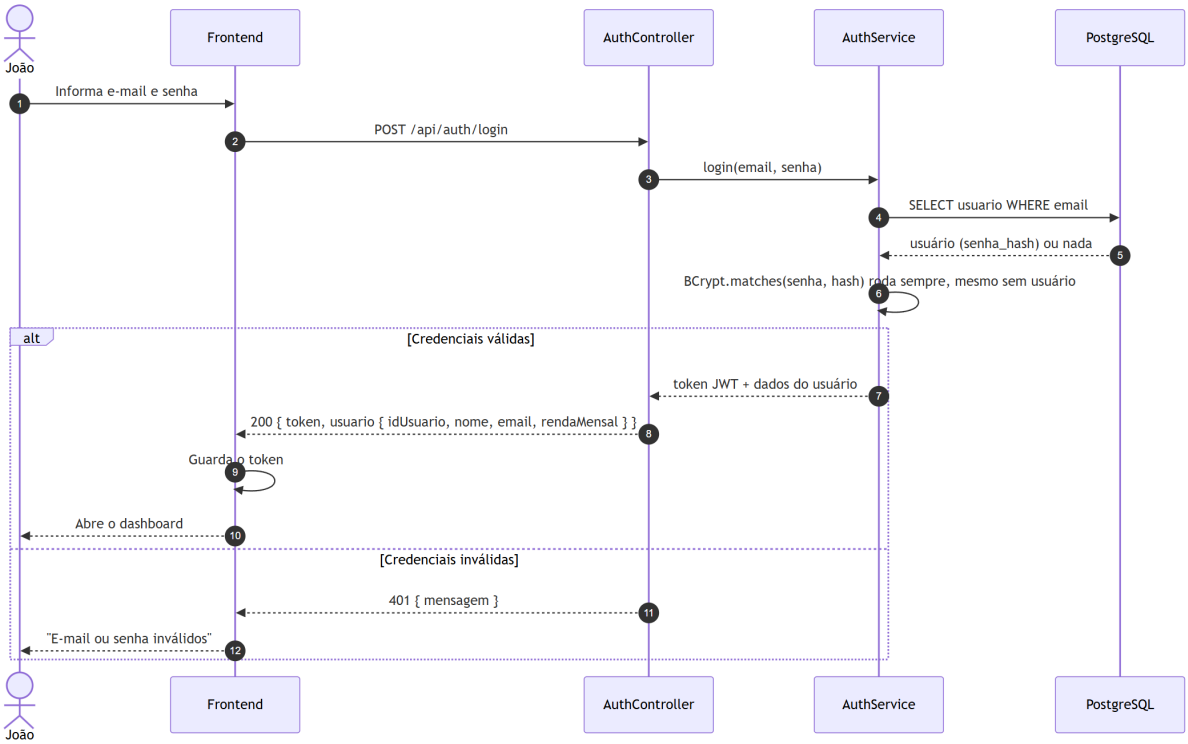
2. Diagrama de classes



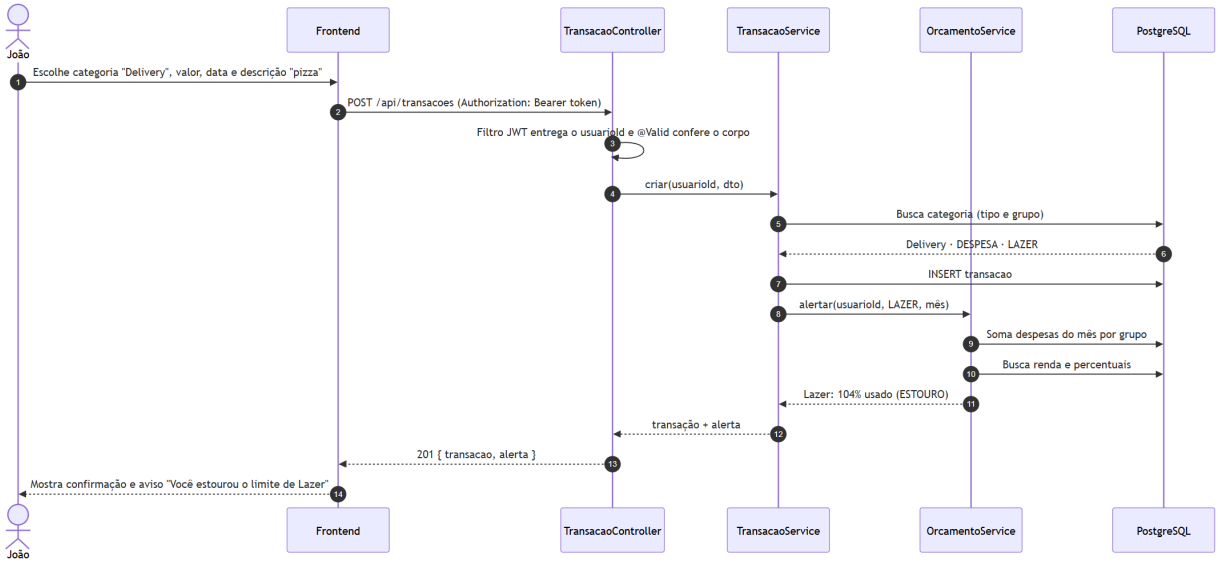
> Os DTOs (`ResumoResponse` , `StatusGrupoResponse` , `TransacaoComAlertaResponse` etc.) não são tabelas. Não existe classe de conta/carteira nem de investimento persistido. O progresso da meta (limitado a 100%) é calculado no `MetaResponse` , não na entidade.

3. Diagramas de sequência

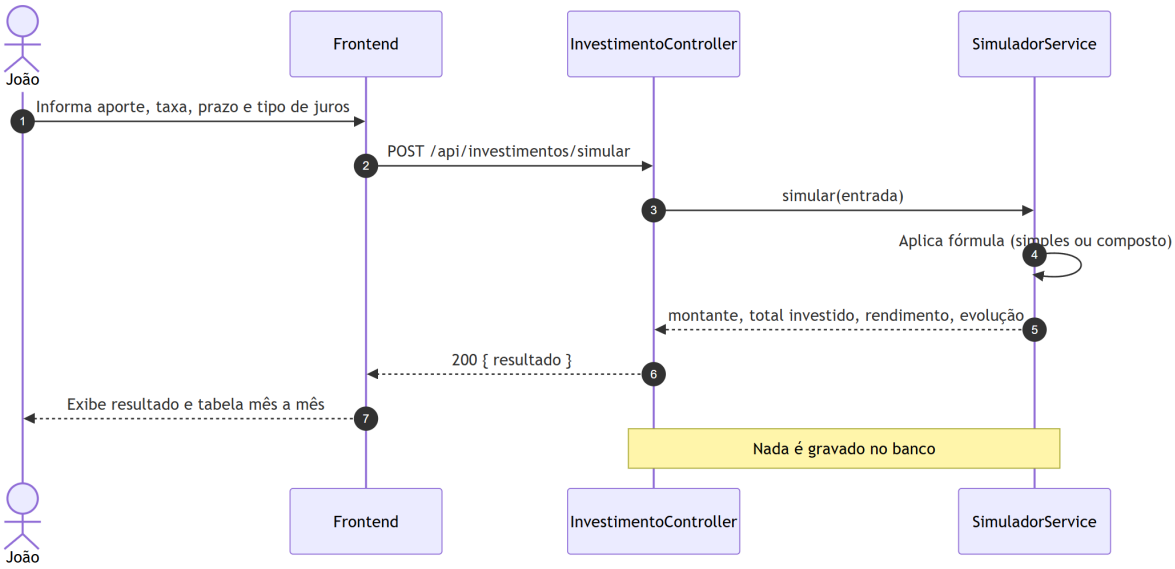
3.1 Login



3.2 Registrar despesa com alerta de orçamento



3.3 Simular investimento (sem persistência)

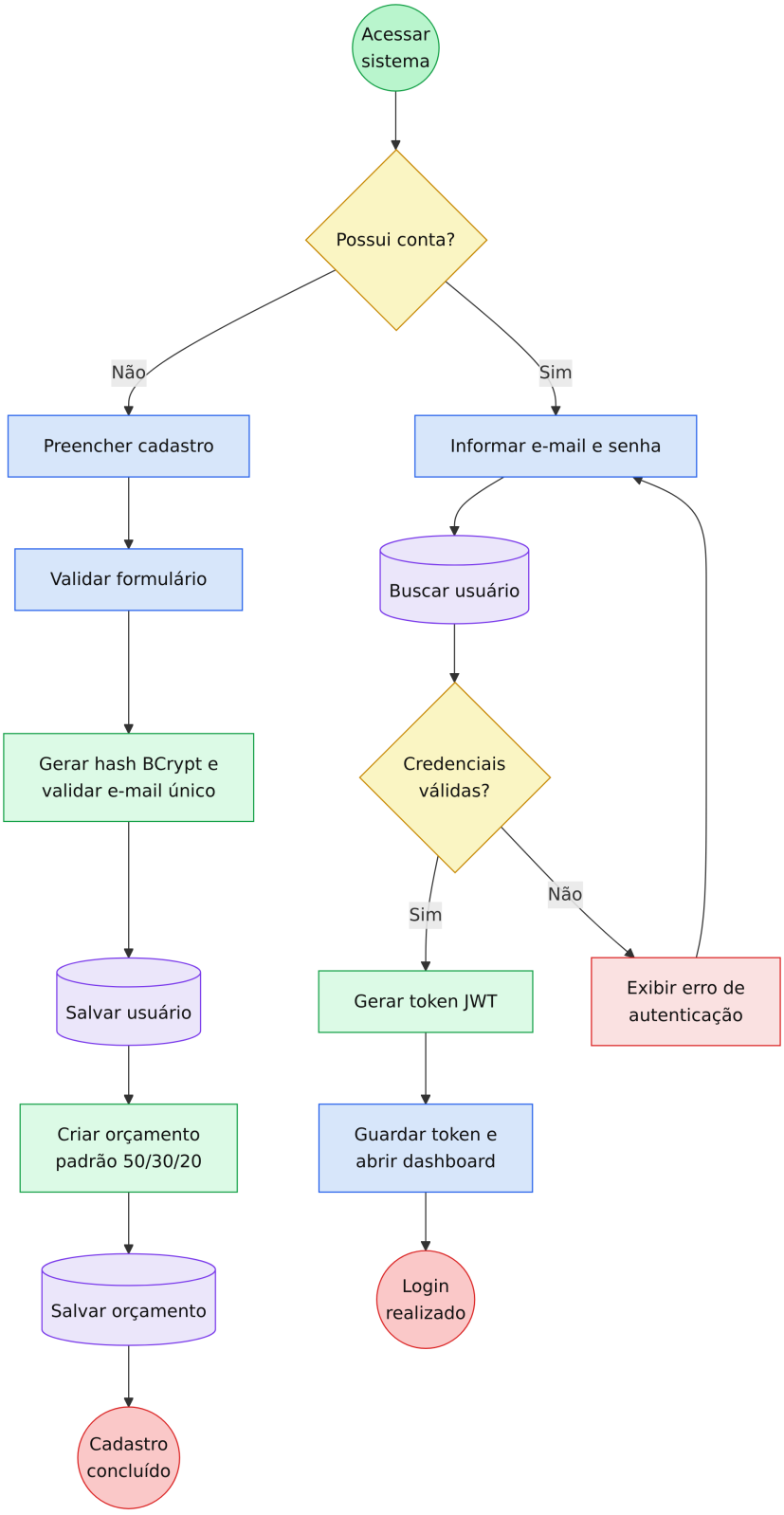


4. Diagramas de processo (BPMN simplificado)

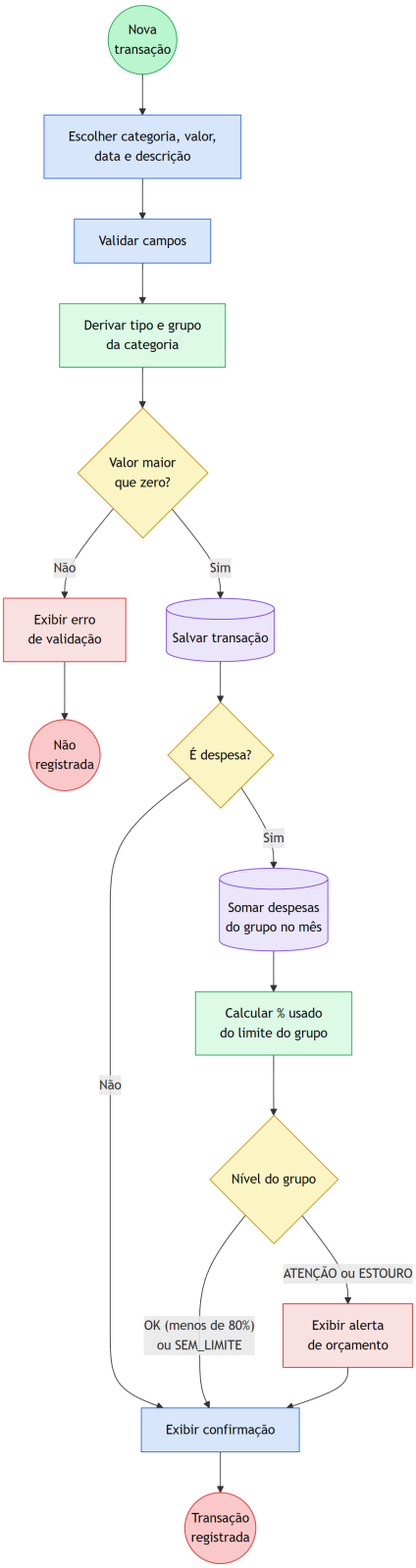
Os processos seguem a legenda da V1: **cor** indica quem executa a tarefa. Formas: retângulo = tarefa, losango = decisão, círculo = início/fim, cilindro = banco de dados.

Cor	Significado
Azul	Tarefa do usuário / frontend
Verde	Tarefa do backend (processamento)
Roxo	Persistência no banco de dados
Vermelho	Mensagem de alerta ou erro
Amarelo	Decisão (gateway exclusivo)

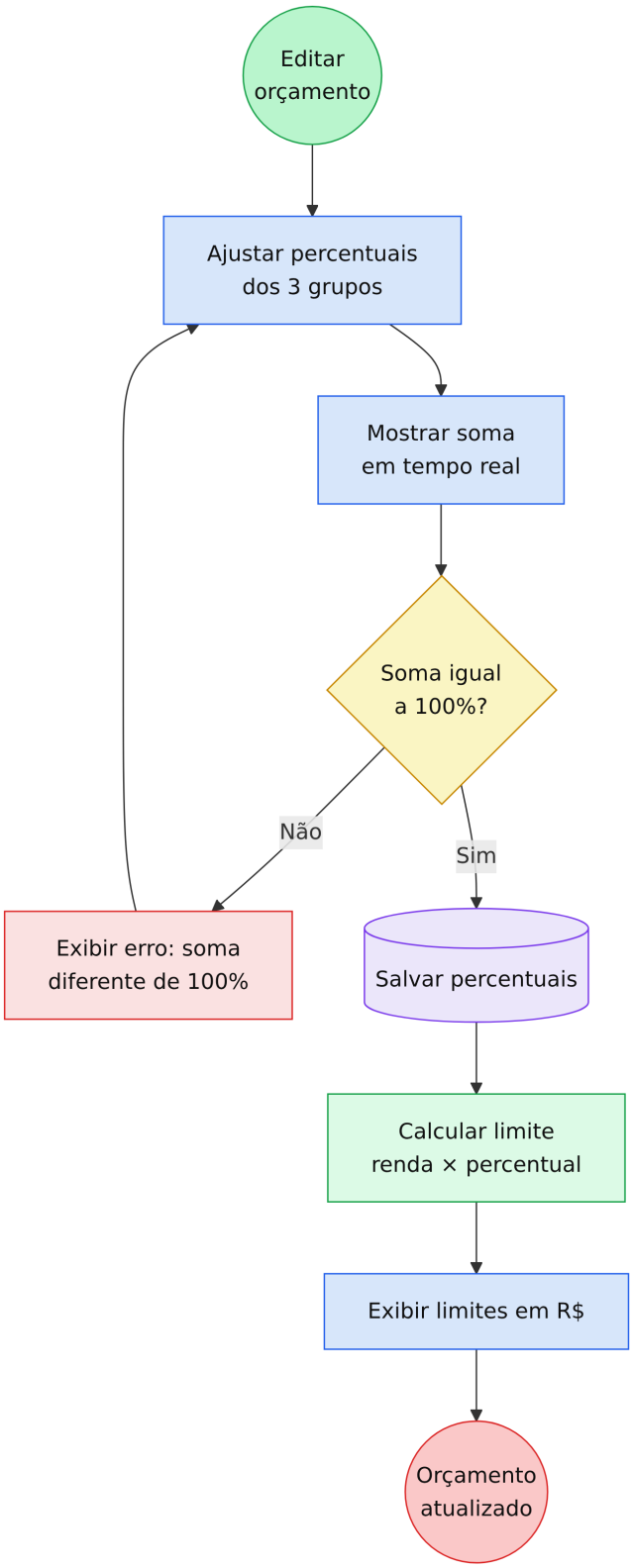
4.1 Processo 1 — Autenticação e cadastro



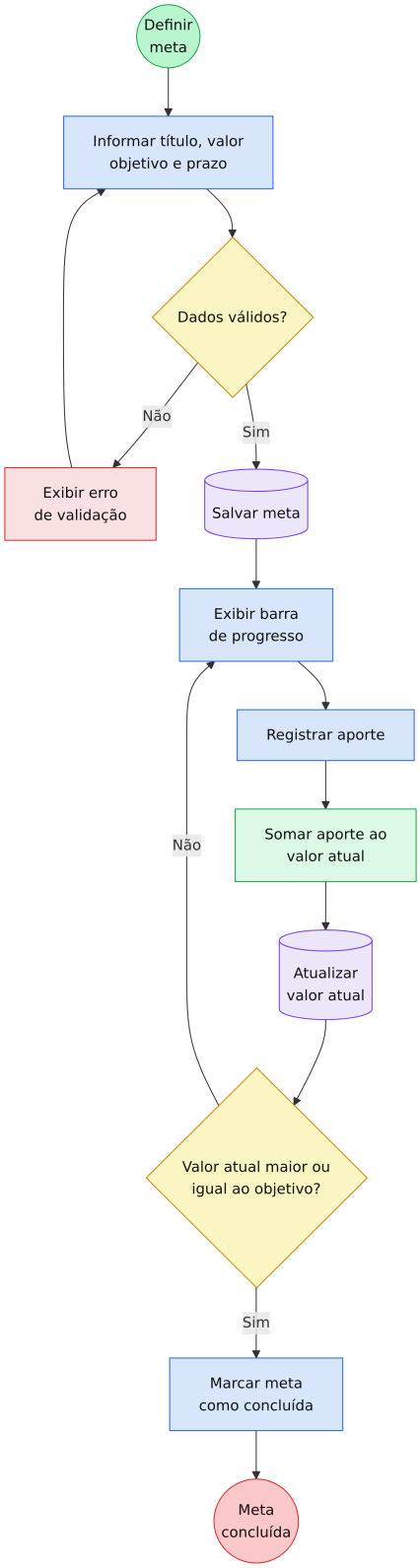
4.2 Processo 2 — Registrar transação e alertar estouro



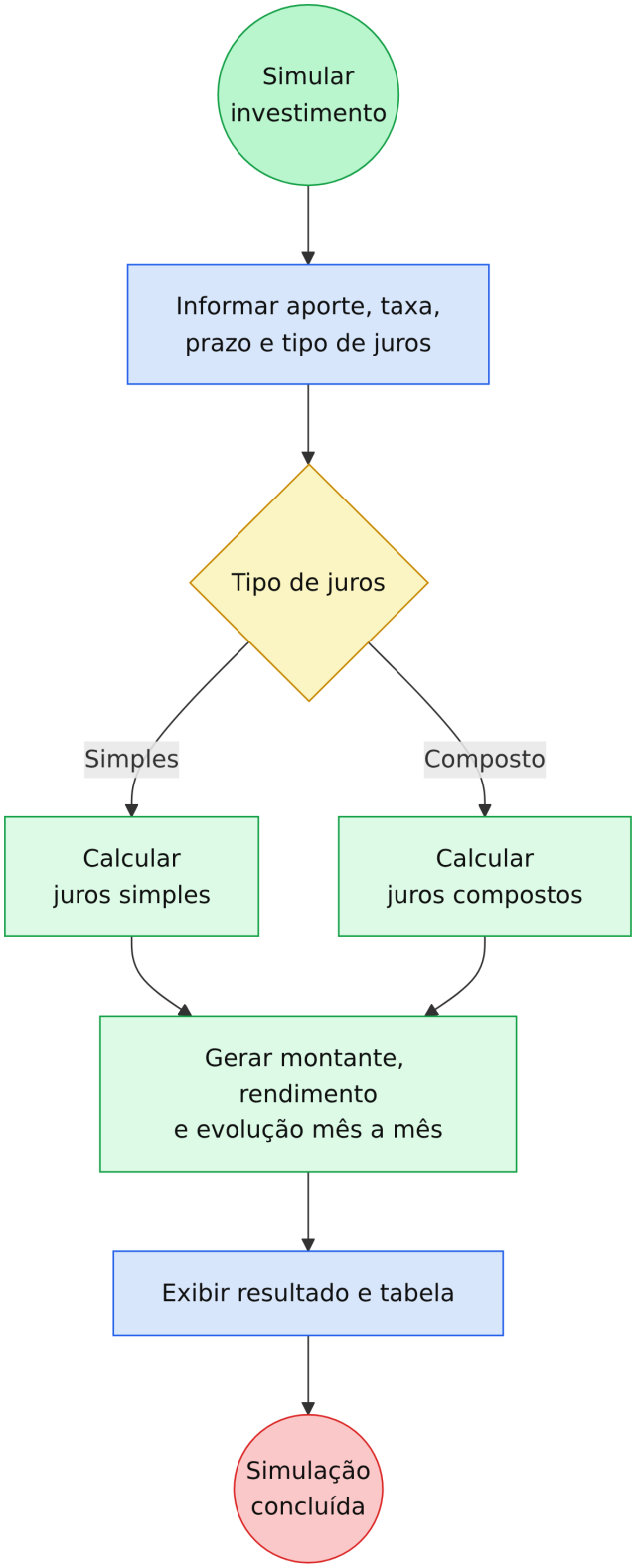
4.3 Processo 3 — Configurar orçamento



4.4 Processo 4 — Metas financeiras

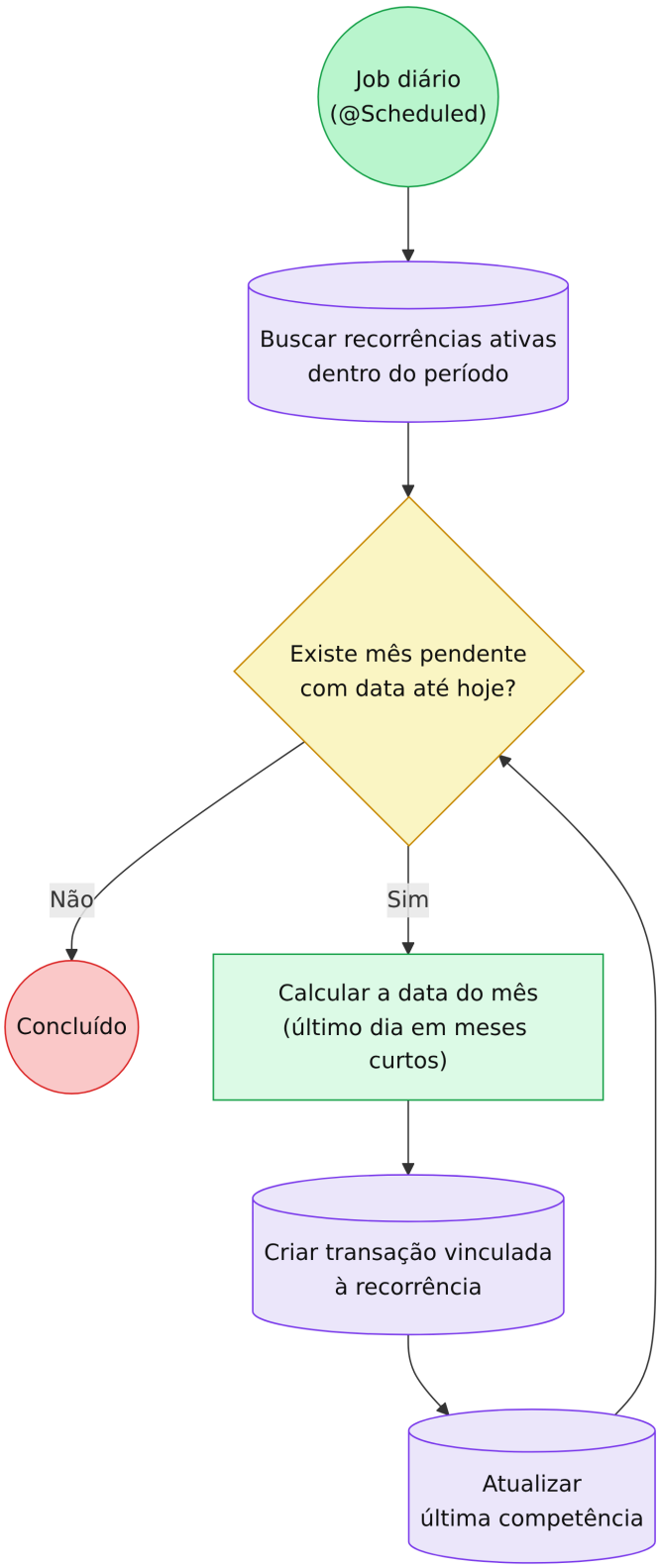


4.5 Processo 5 — Simulador de investimentos



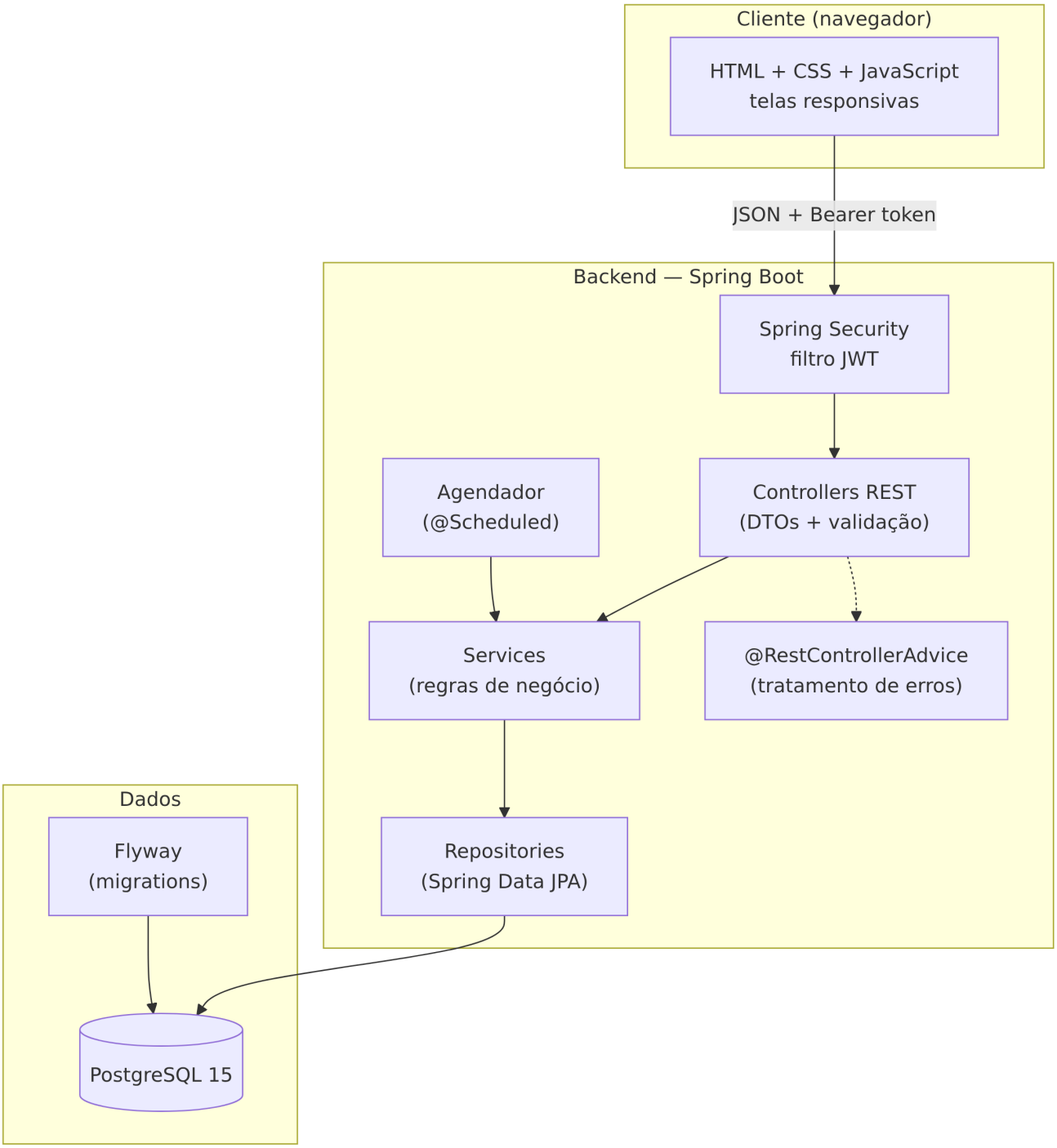
> Observação: a V1 tinha uma etapa "Salvar simulação no BD". Ela foi **removida** (decisão ADR-003).

4.6 Processo 6 — Geração automática de despesa recorrente



> A geração é **idempotente**: a última competência impede duplicar transações mesmo que o job rode duas vezes ou fique dias parado (RN15). O mesmo gerador roda também na hora em que a recorrência é criada ou editada. Pausada não gera, e reativar **não** gera os meses em que ficou pausada (RN17).

5. Arquitetura em camadas



06 · Roadmap, Backlog e Decisões

Roadmap

O trabalho segue sprints de 1 semana. A V1 teve as Sprints 0–4; a refatoração (V1 revisada) continua com o prefixo **R**.

Sprint	Meta	Entregas principais	Estado
R0 · Documentação	Contexto fechado e versionado	Escopo, requisitos, diagramas, README e PDF do seminário	Concluída
R1 · Fundação	Backend alinhado ao banco e seguro	Flyway, entidades novas, BCrypt + JWT, tratamento padrão de erros, variáveis de ambiente, docker-compose	Concluída
R2 · Núcleo financeiro	Transações de ponta a ponta	Categorias fixas, CRUD de transações, filtro por mês, resumo mensal	Concluída
R3 · Regras de negócio	Orçamento, metas, simulador e recorrências	Orçamento editável com alerta, metas com aportes, simulador, despesas recorrentes, exportação CSV	Concluída
R4 · Frontend	Interface nova e responsiva	Tokens CSS, mobile-first, telas refeitas, gráfico de pizza, tema claro/escuro	Concluída
R5 · Qualidade e entrega	Pronto para apresentar	Testes (unitários, integração, Cucumber), revisão de segurança, README final, PDF do seminário	Concluída
R6 · Publicação na web	IFO no ar, de graça	Dockerfile e perfil de produção, Vercel + Render + Neon, animações e fluidez no frontend, guia de publicação no README	Concluída

Backlog

Item	Prioridade	Estado
Cadastro, login e perfil	Must	Entregue
Transações com categoria fixa e descrição livre	Must	Entregue
Filtro por mês e resumo financeiro	Must	Entregue
Orçamento 50/30/20 editável e alerta de estouro	Must	Entregue
Metas financeiras com aportes e progresso	Must	Entregue
Simulador de investimentos (juros simples e compostos)	Must	Entregue
Gráfico de pizza das despesas por grupo	Should	Entregue
Despesas recorrentes (geração mensal automática)	Should	Entregue
Exportar transações em CSV	Should	Entregue
Dark mode	Should	Entregue
Publicação na web (Vercel, Render e Neon)	Should	Entregue
Animações e fluidez no frontend	Should	Entregue

O que vem depois desta versão está na seção 5 (IFO V2).

Decisões de arquitetura

ID	Decisão	Motivo
ADR-001	Categorias fixas, em tabela carregada por migration	Permite chave estrangeira e grupo por categoria; o usuário só edita a descrição
ADR-002	Tipo e grupo da transação vêm da categoria	Evita inconsistência (ex.: "Delivery" marcado como Necessidade)
ADR-003	Simulador sem persistência, calculado no backend	O escopo pede só simulação; com BigDecimal a regra é testável e única
ADR-004	BCrypt + JWT sem estado	Acaba com senha em texto puro e com a passagem do usuário por parâmetro de URL da V1
ADR-005	Flyway e `ddl-auto=validate`	Esquema reproduzível, sem divergência entre SQL manual e entidades
ADR-006	Frontend em HTML, CSS e JavaScript puro, com módulos ES	Mantém a leveza e o controle da V1
ADR-007	Remover Conta/Carteira	Não será usada; simplifica modelo e diagramas
ADR-008	Alerta informativo, atenção a partir de 80%, nunca bloqueia	Pedido: "notificação simples de estouro"
ADR-009	Dashboards interativos adiados para a V2	Controle de escopo
ADR-010	Diagramas em Mermaid versionados junto do código	Documentação sincronizada com o sistema
ADR-011	Despesas recorrentes: modelo mais job diário idempotente	Não duplica transações e recupera meses perdidos
ADR-012	CSV com `;` , vírgula decimal e UTF-8 com BOM	Abre direto no Excel em português; neutraliza injeção de fórmulas
ADR-013	Dark mode por variáveis CSS e preferência do sistema	Sem biblioteca; reaproveita os tokens de design
ADR-014	Frontend multipágina, cliente de API único, sem `alert` nem `innerHTML`	Centraliza token e erros; fecha o XSS da V1
ADR-015	"Calculadora" renomeada para "Simulador de investimentos"	O nome descreve o que a tela faz
ADR-016	Revisão de segurança: login em tempo constante, segredo de exemplo recusado, CSV neutraliza TAB e CR	Não revelar quais e-mails existem; sem limite de tentativas (risco aceito e documentado)
ADR-017	Testes de frontend fora do repositório	Evita ferramenta nova só para testes; a regra de negócio fica no backend testado
ADR-018	Publicação gratuita: frontend na Vercel, backend em Docker no Render e PostgreSQL no Neon	A Vercel não roda Spring Boot e o Postgres gratuito do Render expira em 30 dias; o Neon acorda sozinho. Risco aceito: o servidor dorme e a primeira visita leva cerca de 1 minuto

ID	Decisão	Motivo
ADR-019	Animações e fluidez no frontend (transição entre páginas, esqueletos de carregamento), desligadas por "reduzir movimento"	O site era "rígido" ao trocar de tela; sem biblioteca e sem lógica no cliente

Publicação na web

O IFO está no ar, de graça (ADR-018). Endereço: **https://ifo-organizador-financeiro-frontend.vercel.app**

Conta de demonstração: `demo@ifo.teste` / `demo12345`. É uma conta **pública e compartilhada**, então não coloque dados pessoais nela.

Peça	Serviço	Observação
Frontend (HTML, CSS e JS)	Vercel (plano Hobby)	Site estático, sempre disponível
Backend (Spring Boot em Docker)	Render (plano gratuito)	Dorme após 15 minutos sem uso; a primeira visita leva até 1 minuto e o site avisa
Banco (PostgreSQL)	Neon (plano gratuito)	1 GB; acorda sozinho quando é chamado

Segurança em produção: segredos só nos painéis dos serviços, nunca no Git; a API aceita chamadas somente do site da Vercel (CORS); nenhum dado de demonstração foi carregado na nuvem; erros não mostram detalhes internos.

Próxima versão (IFO V2)

A V2 não acrescenta integração bancária, aplicativo mobile nem sugestões com IA. As principais diferenças em relação a esta versão são duas, mais a troca de tecnologias:

- **Autenticação em dois fatores (2FA):** depois da senha, o usuário recebe um código de 6 dígitos por e-mail (válido por 10 minutos, de uso único).
- **Dashboard interativo:** gráficos com Chart.js (pizza dos gastos, linha da evolução do saldo e barras de receitas × despesas por mês).

Stack do IFO V2

Camada	V1 revisada (esta versão)	V2
Backend	Java 17 + Spring Boot	Node.js + Express, em MVC
Banco relacional	PostgreSQL	MySQL (usuários e códigos 2FA)
Banco NoSQL	não havia	MongoDB (transações, categorias, orçamentos, metas, histórico e análises)
Frontend	HTML, CSS e JS puro	React (Vite) + Tailwind CSS
Gráficos	pizza simples (CSS)	Chart.js
Autenticação	BCrypt + JWT	JWT em cookie httpOnly + 2FA por e-mail (Resend)
Design	sem protótipo	Figma antes de codar cada tela
Acesso a dados	JPA/Hibernate	Sequelize (MySQL) e Mongoose (MongoDB)